Inside the Spec

Real Scenarios. Working Artifacts.
Decisions That Matter.

One Status, Three Systems, Different Meanings

Cross-System Status Mapping
A return workflow with separate evidence for each result.

CASE 02 · STATUS MAPPING

“Completed” does not name the result.

A customer returns one item for USD 80.00. The store, warehouse, and payment provider can all report COMPLETED, while describing different objects.

Source / object	What COMPLETED establishes
OMS · return request	Accepted return processed; refund instruction registered.
WMS · inspection	Inspection finished. The result can be ACCEPTED or REJECTED.
PSP · refund	Provider confirms issuance. Bank credit is not independently confirmed.

> **Mapping each value to “Money refunded” loses the evidence behind it.**

The rejected store workflow has a separate CLOSED_NO_REFUND outcome. In this case, REJECTED means the return was not accepted.

Illustrative case, not a production incident. One item, one inspection, and at most one full refund. USD 80.00 is example data, not a global validation constant.

Keep facts under their owners.

Owned dimension	Allowed projected values
OMS · return_state	UNKNOWN; OPEN; CLOSED_REFUND_REGISTERED; CLOSED_NO_REFUND
WMS · inspection_state	UNKNOWN; PENDING; ACCEPTED; REJECTED
PSP · refund_state	UNKNOWN; PENDING; COMPLETED; FAILED

UNKNOWN means no valid evidence in that dimension. A source snapshot updates only its own dimension. The integration derives a display result from the three facts.

The raw inspection_result exists only for WMS COMPLETED. It must be ACCEPTED or REJECTED. IN_PROGRESS forbids that field; lowercase “accepted” is invalid.

> **A matching PSP event is necessary for REFUND_ISSUED. Warehouse acceptance is also required.**

Case decision: late OMS OPEN does not veto issuance. Missing WMS acceptance does. An early PSP result is preserved while warehouse evidence is checked; a contradictory no-refund closure requires reconciliation.

Evaluate the first matching rule.

O = return_state; W = inspection_state; P = refund_state. “P known” means P is not UNKNOWN. Evaluate D01–D09 in this order.

Condition	Result
D01 · Unresolved reconciliation flag	NEEDS_RECONCILIATION
D02 · Rejection + refund evidence, or no-refund closure + acceptance/refund evidence	NEEDS_RECONCILIATION; latch flag
D03 · W = REJECTED	RETURN_REJECTED
D04 · W unknown/pending + P known or either closed OMS outcome	AWAITING_CONFIRMATION
D05 · W = ACCEPTED; P = COMPLETED	REFUND_ISSUED
D06 · W = ACCEPTED; P = FAILED	REFUND_FAILED
D07 · W = ACCEPTED; P = PENDING	REFUND_PROCESSING
D08 · W = ACCEPTED; P = UNKNOWN	REFUND_NOT_CONFIRMED
D09 · Otherwise	RETURN_IN_PROGRESS

D02 precisely: (W=REJECTED AND (P known OR O=CLOSED_REFUND_REGISTERED)) OR (O=CLOSED_NO_REFUND AND (P known OR W=ACCEPTED)). D04 follows D02, so contradictions take precedence.

Bind identity before judging content.

Every snapshot carries source_system, object_type, object_id, return_id, event_id, revision, contract_version, occurred_at, and a full payload.

Validation boundary	Required response
A · Untrusted identity or linkage	Quarantine; integration incident. Do not change a return fact or flag.
B · Trusted identity, invalid content	Quarantine; retain the last valid fact and revision; latch reconciliation and open a case.

PSP includes refund_id, amount_minor, and currency. refund_id must match the bound object. Amount and currency must exactly match the registered instruction. A wrong amount on the correct refund is B, not an ignored event.

The trusted registry exists before dispatch. Contract version is 1. Values are case-sensitive; required fields cannot be null. Unsupported versions on reliably bound objects are B.

Native provider identifiers may differ from internal IDs. The adapter verifies their binding. If the provider lacks monotonic full snapshots, the adapter must establish authoritative ordering; arrival order alone cannot define revisions.

A replay and an older revision differ.

Incoming snapshot	Decision
Same source + event_id, identical event	Return the recorded outcome; no duplicate action.
Valid revision below accepted baseline	Ignore as stale.
Equal revision, equal business payload	No-op even if event_id differs.
Equal revision, different payload	Keep fact; latch reconciliation.
Higher revision	Validate allowed transition; commit atomically.

Worked order, with WMS ACCEPTED: PSP PENDING rev 1 → COMPLETED rev 4 → first delivery of PENDING rev 3. The last event is stale; REFUND_ISSUED remains.

An identical replay of the rev 4 event is a deduplication no-op. Revision gaps are permitted because snapshots are complete. Revisions are comparable only within one source object.

> **Serialize updates per return or retry a projection-version conflict. One source must not erase another source’s fact.**

Content validation precedes revision comparison. A malformed old event still raises a linked validation issue. Ordinary delayed events do not require re-baseline.

Issued is not an irreversible success.

PSP COMPLETED is nonterminal. A newer, valid FAILED snapshot is a normal late outcome, even days later. It replaces the provider fact without manual re-baseline.

PSP transition	Handling
PENDING → COMPLETED	Issuance confirmed; continue late-outcome monitoring.
PENDING → FAILED	Failure confirmed; open a payments case.
COMPLETED → FAILED	Accept late failure; create durable finance handoff.
COMPLETED → PENDING	Invalid transition; reconciliation.

Internal result	Customer-facing wording
REFUND_ISSUED	Refund issued by provider. Your bank may not show the credit yet.
REFUND_FAILED	Refund needs attention. The provider reported a refund failure.
NEEDS_RECONCILIATION	Return status being checked.

A late failure is retained even when another conflict masks it in the UI. It still creates a payments case and finance handoff. Provider issuance, bank visibility, and ledger effects are separate facts.

A crash leaves uncertainty to resolve.

OMS commits refund intent, linkage, and the outbox row in one transaction. The worker records each attempt before calling PSP and uses the same instruction and idempotency key.

Dispatch state	Required behavior
READY → IN_FLIGHT	Acquire lease; persist attempt before create.
Provider response or linked webhook	Bind native refund ID; record ACKNOWLEDGED. This is not payment success.
Timeout or expired IN_FLIGHT lease	Move to UNCERTAIN; query the original operation.
Existing provider object found	Acknowledge it; do not create another refund.
Guarantee missing, expired, or retry limit reached	MANUAL_REVIEW; preserve uncertainty.

A temporary “not found” is not proof of non-execution. Create retries require identical parameters and a verified provider guarantee covering scope, retention, and concurrent calls.

Transactional Outbox preserves intent; it does not establish exactly-once execution at PSP. Persist attempts, request hashes, provider references, lease tokens, retry limits, and dedupe_valid_until. Never rotate a key to escape an ambiguous outcome.

Correct the fact. Keep the obligation.

D02 means contradictory evidence, not missing evidence or a fraud finding. A final warehouse rejection and a refund remain contradictory in either delivery order.

Re-baseline step	Constraint
Fetch authoritative source snapshots	Verify identity, full payload, amount, and currency.
Validate revision and correction	No lower baseline. Equal revision with changed data requires source-owner confirmation.
Replace fact and baseline atomically	Audit operator, reason, evidence, before/after values. Recheck concurrency.
Recompute all contradictions	Do not clear D02 with a comment alone.

Example: authoritative WMS REJECTED rev 3 can replace ACCEPTED rev 2. If PSP issuance still conflicts, the corrected warehouse fact is stored and reconciliation remains open.

> **A financially consequential correction also persists a finance-outbox record. ERP downtime must not block the truthful fact.**

Closing reconciliation requires an accepted finance handoff where applicable and no remaining D02 conflict. Finance determines postings, compensation, or a no-posting decision. Re-baseline itself neither moves money nor creates a customer debt.

Use deadlines that match the stage.

Condition	Deadline and owner
Missing integration evidence	Shared 15-minute interval for AWAITING_CONFIRMATION / REFUND_NOT_CONFIRMED; integration operations.
PSP PENDING	Pinned provider/method processing deadline; payments operations.
PSP COMPLETED	Pinned visibility-check deadline; keep monitoring late outcomes.
PSP FAILED or reconciliation	Immediate owned case; financial handoff where required.
RETURN_IN_PROGRESS stalls	24-hour example threshold; returns operations.

Payment policies include provider/account, method, region, currency, calendar, timezone, version, and absolute UTC deadlines. No matching policy blocks dispatch; there is no generic processing fallback.

Provider timers start from first valid source evidence, even when a different UI state masks it. Repeated snapshots do not reset them. Switching within the shared evidence-wait group does not reset its timer.

The fixed 15-minute, 24-hour, and four-hour case-review intervals are illustrative assumptions. Open cases escalate to the owning lead after four hours and receive another review deadline. A handoff requires an accepting owner and case reference.

Test meaning as well as ordering.

Acceptance scenario	Expected behavior
AC-15 · Correct refund, wrong amount	Keep the valid fact; latch reconciliation; open a case.
AC-29 · Rejected item, no-refund closure	RETURN_REJECTED; no false conflict.
AC-69 · Failure after issuance	Accept the newer FAILED; payments case and finance outbox.
AC-76 · Crash after PSP acceptance	Find and bind the existing refund; no new create.
AC-81 · ERP unavailable during correction	Persist the correction and finance handoff; retain reconciliation.

> **88 acceptance scenarios specify expected behavior. They are not executed integration tests.**

The finite model checks 64 state tuples plus 64 reconciliation overrides. It verifies projection totality, selected invariants, and nine manually specified outcomes.

These checks do not validate delivery, concurrency, dispatch, provider guarantees, calendars, timers, or ERP integration. Source contracts and operational policies must be verified before production reuse.

Beyond the Scope

Extension	What needs a separate model
Partial refunds / multiple items	Per-item decisions, multiple refund objects, parent aggregation, and concurrent balance limits.
Fees, taxes, shipping deductions, FX	Explicit amount components and currency-specific invariants.
Chargebacks / new refund attempts	Separate objects, authority, transitions, and financial handling.

Per-refund mappings may be reusable where source meanings match. D01–D09 must not be copied as the parent-level rule for a multi-item order.

Review question: which system owns the evidence required for the status we show the customer?

Full specification – see GitHub.

GitHub Repository for the Inside the Spec

Yulia Melekhova on LinkedIn